

AdaptiveDonutOCR

Learned visual-token pruning for document OCR

Nafis Kamal

Independent research project · Technical report, October 2026

<https://github.com/nafis8766/adaptive-donut-ocr>

Donut renders a document page as 4,800 visual tokens, most of which are blank paper. This project inserts a learned scoring head between the frozen encoder and the decoder, ranks those tokens, and discards the low-ranked ones before the decoder attends to them. Two questions: **how few can you keep**, and **does learning which ones to keep beat not learning it**. Three results stand, each stated with the scope limit it carries:

1. **Discarding 65% of the visual tokens frees 65% of the decoder’s cross-attention KV cache for -0.26 points of word recall** ($t = -0.18$, $n = 50$ paired) — a measured null, not an absence of measurement. The condition is load-bearing and was tested: this holds *only* for a checkpoint trained with pruning active. The same operation on a model trained unpruned costs 6.03 points at the same budget.
2. **Learned selection is worth $+17.7$ to $+34.0$ points of word recall** over a random mask at an identical token budget, with the margin widening as the budget tightens.
3. **The gain is train/test matching, not inference-time denoising.** The competing hypothesis was written down in advance, given its most favourable condition, and rejected by a difference-in-differences design on bit-identical token sets.

What the work does **not** show, stated here rather than in a footnote: no latency or throughput win, no encoder-side saving, and no resolved result for the token-merging stage. Section 6 gives each of these its measurement.

1 Problem and approach

Model. `naver-clova-ix/donut-base`. The Swin-B encoder emits an 80×60 token grid at 2560×1920 input (stride 32), so 4,800 patch tokens reach the decoder. The decoder is 4 layers $\times$ 16 heads $\times$ 64 head-dim, $d_{\text{model}} = 1024$; parameters occupy 771 MiB at fp32.

Router. A two-layer MLP scoring head with a straight-through estimator, $M_{\text{ste}} = \text{TopK}(s) + s - \text{detach}(s)$: hard discrete filtering in the forward pass, end-to-end gradients from the decoder’s loss back into the scorer. The router sits *after* the frozen encoder, which is precisely what bounds the efficiency story to the decoder side — see Section 6.

Task and data. Full-page OCR into a single `{"text": ...}` target. FUNSD (`nielsr/funsd`; 149 train, 50 test) plus 500 SynthDoG-en pages. SROIE is deliberately excluded from training: its key-information-extraction schema would pollute a single-field target.

Evaluation protocol, held constant across every run: FUNSD test $\times$ 50 images; word recall, character accuracy, word order, mean normalised edit distance; `MAX_WORDS=128`; `max_length=512`; task prompt `<s_doc>`; `repetition_penalty=1.0`, `no_repeat_ngram_size=3`.

Selection modes. Evaluation can swap the *ranking signal* while holding the token budget fixed: `router` (learned scores), `random` (uniform), `ink` (pixel contrast, no learning), `negated` (router scores inverted), and two per-grid-row stratified variants. Because only the ranking changes, rows are comparable at equal cost. `ink` is additionally *weight-independent* — it is a function of the image alone — which is what makes it usable for comparison across checkpoints (Section 4).

Two venues, which do not mix. Training runs on Kaggle T4s; diagnostics run locally on CPU against cached checkpoints. The same weights on the same 50 images score 77.74 on Kaggle and 74.72 locally — different device, different `transformers` version. Token *selection* reproduces across the two to three decimal places; *generation* does not, and the gap widens as the budget tightens. **Every figure in this report is therefore a within-checkpoint, within-venue paired delta.** No number compares a local level to a remote one.

2 Result 1 — the budget claim

The checkpoint is trained with pruning active at `keep_ratio=0.50` for five epochs, resumed from a model trained unpruned. Evaluated at four budgets against its own unpruned ceiling, on the same 50 images, paired per image:

keep	visual tokens	cross-attn KV	word recall	Δ vs unpruned	t	verdict
1.00	4800	150.00 MiB	77.62	—	—	ceiling
0.50	2400	75.00 MiB (−50.0%)	78.22	+0.60	0.43	free
0.35	1680	52.50 MiB (−65.0%)	77.37	−0.26	−0.18	free
0.25	1200	37.50 MiB (−75.0%)	70.67	−6.96	−2.82	costly
0.20	960	30.00 MiB (−80.0%)	64.61	−13.01	−5.42	costly

The headline is the `keep=0.35` row: 65% of the tokens gone, 65% of the cross-attention KV freed, no detectable accuracy cost. The two percentages are the same percentage — cross-attention KV is linear in retained tokens — which is why this can be one sentence rather than a rate and a price the reader has to trade off.

Three properties make it a result rather than a hope.

It is powered. A flat row means nothing if the experiment could not have resolved a real effect. The paired standard error puts the minimum detectable effect at **2.76 and 2.81 points** on the two free rows, against a floor of 3.0 fixed in advance. So “we looked and saw nothing” is distinguishable from “we could not have seen it,” and this is the former. The two tight budgets are reported as costs rather than quietly dropped.

The memory figure is measured twice, by independent routes. It is computed analytically ($2 \times \text{layers} \times \text{heads} \times \text{head_dim} \times \text{tokens} \times 4\text{B}$) and captured from the real `past_key_values` through a forward hook on the decoder. The two agree to a **relative gap of 0.0000** at all five budgets, with a control asserting that the captured tensor’s sequence length equals the number of compressed tokens `generate()` reports. Self-attention KV — which pruning does not touch — is separately asserted to be a function of generated length only, and is *non-monotone* in the keep ratio, so a self/cross mix-up could not be mistaken for a pruning win. Twenty-five of twenty-five controls pass.

The headline budget was fixed by a rule, not chosen by inspection. `keep=0.20` gives a larger memory cut (−80.0%) and a better-sounding number; it costs 13.01 points. The measurement script pins the headline to the tightest budget that is *free* and fails a control if that budget turns out not to be, so the framing cannot drift toward the largest available percentage.

Scope. Cross-attention KV only — not total memory, not peak, **not the encoder** (the frozen Swin computes all 4,800 tokens at every budget), and not latency. Unpruned cross-attention KV is **19.46% of the model’s 771 MiB of parameters**, so this is a large cut in a real quantity, but it is one quantity. All byte figures halve at fp16 and scale with batch size; the *percentage* is invariant to both.

3 Result 2 — the value of learned selection

Random selection at the same budget is the floor a learned router has to clear. Paired on the same 50 images, with **random** reseeded per configuration so every comparison sees identical masks:

keep	visual tokens	router	random	margin
0.50	2400	80.46	62.77	+17.7
0.35	1680	79.25	51.84	+27.4
0.25	1200	71.15	37.20	+34.0
0.20	960	63.10	33.80	+29.3

The margin *grows* as the budget tightens, up to `keep=0.25`, then falls back slightly at `keep=0.20` where both arms are badly degraded. That shape is the expected one: at a slack budget a random subset still catches most of the text and there is little for a selector to add; at a tight budget every retained token has to carry glyphs.

This margin was not free to obtain, and the reason is worth recording because it is a general trap. For five runs the router was an **inverted** saliency detector — it ranked blank paper *above* text. The cause is structural: training a top- k router at `keep_ratio=1.0` makes the straight-through multiplier exactly 1.0, so the only gradient the scorer ever receives asks “*does scaling this token’s magnitude help*” rather than “*is this token informative*”, and those two objectives anti-align. The symptom was that inverting the router’s own scores at evaluation time scored 73.09 while the forward router scored 18.80 — the ranking carried strong signal with the wrong sign. The fix was an explicit sign-correct supervision term on the scoring head; the confirmation was a role flip, with **negated** collapsing from 73.09 to 2.70 afterwards.

Scope. This is a claim against *random* selection. It is **not** a claim that the router beats a hand-crafted ink heuristic: at `keep=0.50` the router scores 2.24 points above ink-oracle selection while retaining 0.072 *less* of the page’s ink — so it is not merely rediscovering ink — but the paired margin is $t = +1.62$ on $n = 50$, under the threshold of 2.0 fixed in advance. Directional, not significant, and reported as such.

4 Result 3 — why it works, tested rather than assumed

For a week the project recorded that pruning *helped* at some budgets and that why was unknown, with “some regularisation-like effect” as the standing guess. Two hypotheses were written down before the test:

- **H1, train/test matching (mundane).** The checkpoint trains at `keep=0.50`, so evaluating it unpruned is off-distribution. Its unpruned “ceiling” is depressed, and the apparent gain from pruning is a *recovery* toward its own operating point.
- **H2, inference-time denoising (interesting).** Most of a page is blank paper, which dilutes cross-attention. If that is the mechanism, pruning should also help a model that never trained with it.

A second checkpoint — trained *and* evaluated unpruned — discriminates between them. H2 predicts its curve improves under pruning; H1 predicts it peaks unpruned and falls. Both checkpoints were evaluated with **ink** selection, which is weight-independent, so **both keep bit-identical token sets** (verified at a gap of 0.00e+00 on all four budgets). Every difference between the two curves is therefore attributable to the weights.

keep	Δ , trained unpruned	Δ , trained at 0.50	difference-in-differences		t
0.50	−1.57	+0.60	−2.17		−0.81
0.35	−6.03	−0.26	−5.77		−2.28
0.25	−11.47	−6.96	−4.52		−1.52
0.20	−20.34	−13.01	−7.33		−3.00

H2 is rejected. The unpruned-trained model declines monotonically where H2 predicted improvement, and the pruning-trained model peaks at exactly the budget it was trained for. H2 was moreover given its *most favourable* condition and still failed: the ink oracle retains **0.994** of the page’s ink at `keep=0.50`, so what was discarded really was blank paper — precisely what H2 says should help.

So the deliverable requires the word “if”. Pruning to a third of the visual tokens costs nothing *if you train for it*. It is not a property of the pruner.

The confound was named before the run and then removed by a dedicated arm. The two checkpoints above differ by pruning-aware training *and* by five more epochs of it, so the comparison established that a difference exists, not what caused it. The isolating run is the same configuration with `TRAIN_KEEP_RATIO` moved 0.50 $\rightarrow$ 1.00 and nothing else changed, verified by diffing the executed notebooks — the five epochs *without* the pruning. Measured paired on $n = 50$:

- **ISO(0.35) = −10.49 points** ($t = -3.74$): removing the pruning while keeping the epochs costs ten points.
- That run’s budget curve **peaks unpruned and declines monotonically**, where the pruning-aware curve is flat.
- Five unpruned epochs moved the ceiling by +0.52 points ($t = 0.24$) — the epochs alone buy nothing.

So the gain is attributable to **pruning-aware training**, not to the extra epochs. **One limit stands, and it is why the wording is precise:** the isolation holds at $\text{keep}=0.35, 0.25$ and 0.20 ($-8.15, -7.35, -7.92$ locally, every t at or beyond -2.70). The *loose* budgets are **underpowered, not null**. The defensible sentence is “H1 isolated at $\text{keep}=0.35/0.25/0.20$ ” and never “H1 proven”.

5 Method — how these claims are defended

The verification layer is a deliberate part of this project rather than scaffolding around it, and several of its most useful findings came from there rather than from GPU time.

Diagnostics re-derive their numbers from cached artifacts, so any claim can be rechecked on a CPU without a training run. Fourteen numbered diagnostics plus a memory measurement are each a standalone, seeded script that prints an explicit verdict and a concern list. Several falsified the hypothesis they were built to support: the first inverted the plan it was meant to enable, the second overturned the first’s recommendation in under a second of compute, and the “why does pruning help” study rejected the guess the project had carried for a week. The memory script’s pre-written framing — “a large cut in a small quantity” — was falsified by its own output at 19.46%, and the script now derives that sentence from the measurement instead of asserting it.

Patches are shown to run, not to parse. Every notebook edit goes through a patcher that backs up first, asserts each anchor matches exactly once, and parses the result; every patcher has a companion that *executes the real cell* against stubs, with no GPU and no weight download. The training loop, the loss block, the mid-run weight-swap control and the results writer are all covered this way. Three correctness defects were found here rather than by a run failing: a decoding setting that silently capped output at 71% of gold length, the `keep_ratio=1.0` straight-through trap of Section 3, and a loss that fed a probability to a logits-expecting cross-entropy, attenuating the gradient up to $112\times$ on exactly the tokens the scorer was most wrong about.

Green checks get the same suspicion as red ones. Before a passing check is trusted, the question asked is what a *failing* system would score on that same check; if the answer is “about the same”, the check is decorative and gets rebuilt. The clearest case: a router probe printed “no concerns” across five sections, and a sixth section that compared the router to the *actual objective* reversed the verdict entirely. The five clean sections were all measuring properties a broken router also satisfies. Controls are correspondingly chosen so that they can fail — one acceptance gate was moved off a noisy recall delta and onto a near-deterministic quantity, and a mid-flight anchor was pinned to *a number that did not yet exist* and then reproduced it at a gap of 0.00.

A pre-registration was written for a run that was never collected, and it retired this project’s own headline claim. The analysis rule for the token-merging question — estimator, standard error, interval method, a tail statistic alongside the location statistic, multiplicity treatment over three quantities, and a verdict table stating what each verdict licenses — was fixed in advance on the *variance structure* of the paired per-document differences, never on the sign of any contrast. The run it was written to score never happened, so it was instead applied *backwards* to the two completed runs, and the verdicts did not survive it: two figures the project had been reporting as results turned out to reproduce exactly in arithmetic while failing as *verdicts*, one because its sign flips with both the estimator and the replicate, the other because it is unreplicated across the two runs. The rule also found a defect in itself — its tail gate fires on 13 of 21 contrasts in which nothing is merged at all, so it is sensitive but not specific to the mechanism it was written to police. A pre-registration that only ever confirms is not doing anything; this one cost the project its headline, which is the evidence that it was real.

6 Limitations and scope

The token-merging stage is unresolved, and it is reported as unresolved. The architecture includes a bipartite token merger (cosine soft-merge on a checkerboard split of the retained grid). Its only defensible estimand is token-matched: *at an equal budget, does merging preserve more word recall than simply pruning harder?* At a matched budget of $M = 1920$ visual tokens the pre-registered primary contrast measures $+1.66$ $[-0.81, +4.14]$ in one run and $+0.02$ $[-1.90, +1.94]$ in the other, with **0 of 3** confirmatory quantities surviving Holm correction in either. Both are flat, and both sit well above the 1.0-point resolution the design targets — so the honest verdict is **underpowered**, which licenses nothing: not “merging is free”, not “no measured cost”, not even “merging changes nothing”. Reaching the target resolution requires $n \gtrsim 307$ documents. **FUNSD’s test split is 50 documents, and that**

is the entire split, so n was never a parameter to turn.

The pooled evaluation that would have reached $n \approx 410$ was built and verified — a 29-row sweep including a structure-matched random arm at $M = 1920$ that the completed runs structurally could not supply, with cost measured at 5.92 h against a 9 h budget — and was not run, for one reason: no second corpus with licensing that could be established. The first-party-licensed candidate annotates only a key-value subset of each page rather than the page itself, making its recall a different quantity at a measurement grain $4.2\times$ coarser than the effect being resolved; the candidate with the right annotations carried only an uploader-asserted licence with no attribution to the competition that produced the scans. The project stopped at data it could not account for rather than publish a number resting on it.

No latency or throughput win, and this is structural rather than disappointing. Cutting 4,800 visual tokens to 960 — a $5\times$ reduction — buys $1.04\times$ wall-clock, with a maximum of $1.05\times$ across ten measured rows. The router and the merger both sit *after* the frozen Swin encoder, so all 4,800 tokens are computed at every budget, and generation is decoder-bound at 243–280 autoregressive steps against an encoder that runs once. **Any latency or throughput claim from this architecture is false**, and an earlier framing of this work that leaned on one was retracted.

No encoder-side saving, for the same reason, and the quantity is zero rather than small. A per-stage analysis of the encoder’s own configuration places the merger downstream of *all twenty* encoder blocks, so its encoder saving is 0% by construction. There is also no cheap fix: inserting it one stage earlier caps the saving at 4.9% even at a 50% merge ratio, because only 9.7% of the encoder lies downstream of that point. The version that would pay must precede stage 2, which holds 69.2% of the encoder in 14 of its 20 blocks, inside a frozen pretrained encoder whose window partitioning, shifted-window cyclic shift and relative position bias all assume an intact spatial grid. That is a separate project, and it is specified rather than attempted.

The one efficiency claim is narrow by construction. The memory result is cross-attention KV only. Of the $2.50\times$ reduction available at $M = 1920$ with pruning and merging combined, **pruning supplies $2.0\times$ and the merge step alone $1.25\times$** — so the combined ratio is not a merging result and is not quoted as one.

Remaining limitations, each measured and none of them chased:

- $n = 50$ **test images**, which drives the ≈ 2.8 -point resolution on the free rows. Sufficient for the effects claimed, and stated wherever an effect is near it.
- **Local and remote generation diverge**, widening as the budget tightens. This is characterised, and is the reason every figure here is a paired within-venue delta rather than a level.
- **Roughly 18% of predictions are still not valid JSON.** Measured as a rate only and never characterised; predictions are deliberately never repaired before scoring.
- **The unpruned ceiling moves between retrains** — 1.09 and 1.30 points on two runs, and 10.93 on an earlier one where the drift could not be separated from a harness fault. This is why every comparison is within-checkpoint, and why each run from that point on re-evaluates unchanged reference weights through the same harness before anything else. The two early runs stay ambiguous retroactively: no control can be added to a run that already happened.
- **The merger’s ordering fix is correct about the partition and unproven about recall.** Deriving the bipartite split from the original raster index rather than from score rank takes missed vertical redundancy from 100% to 0%, but the head-to-head against the previous split is null on three checkpoints. It is kept on the design argument, and is described as a free parameter rather than a validated setting.

One number to take away. At keep=0.35, a checkpoint trained for it gives up 65% of its visual tokens and 65% of its decoder cross-attention KV cache for -0.26 points of word recall (paired standard error 1.41, $n = 50$). The same operation on a checkpoint not trained for it costs 6.03 points. The distance between those two sentences is the result.

Every figure above traces to a versioned artifact in the repository, and the diagnostics behind them re-run on CPU without a GPU. This report is a dated snapshot; AGENTS.md is the source of truth.